

Data Structures

CSC 301

Lecture Notes

Chapter 5: Tree Data Structures

Computer Science
Osun State University
2026/2027

Prepared by
Tunde Sardine

Contents

5	Tree Data Structures	2
5.1	Introduction to Tree Data Structures	2
5.1.1	Tree Terminology and Properties	2
5.1.2	Tree Traversal Algorithms	2
5.1.3	Implementation Strategies for Trees	3
5.1.4	Applications of Trees	3
5.2	Conclusion	3

Chapter 5

Tree Data Structures

5.1 Introduction to Tree Data Structures

Trees are hierarchical data structures composed of nodes connected by edges, with a single root node and child nodes forming subtrees. Unlike linear structures such as arrays or linked lists, trees allow efficient representation of hierarchical relationships, making them essential for modeling real-world systems like file systems, organizational charts, and decision trees. This chapter explores tree definitions, traversal algorithms, and implementation strategies for binary trees, binary search trees (BSTs), and balanced trees.

5.1.1 Tree Terminology and Properties

A tree consists of nodes connected by edges. Key terms include: - **Root**: The topmost node with no parent. - **Leaf**: A node with no children. - **Internal Node**: A node with at least one child. - **Subtree**: A tree formed by a node and its descendants. - **Depth**: The number of edges from the root to a node. - **Height**: The maximum depth of any node in the tree.

Trees can be classified by constraints on node degrees. A **binary tree** restricts each node to at most two children (left and right). A **full binary tree** has all nodes with 0 or 2 children. A **complete binary tree** fills all levels except possibly the last, which is filled left to right.

5.1.2 Tree Traversal Algorithms

Traversal algorithms visit nodes in specific orders:

- **Preorder (Root, Left, Right)**: Visit the root, then recursively traverse the left and right subtrees.
- **Inorder (Left, Root, Right)**: Traverse the left subtree, visit the root, then the right subtree. For BSTs, inorder traversal yields nodes in ascending order.

- **Postorder (Left, Right, Root):** Traverse left and right subtrees before visiting the root. Useful for deleting nodes or evaluating expressions.

Example: For a tree with root A, left child B, and right child C, preorder traversal visits $A \rightarrow B \rightarrow C$, while inorder visits $B \rightarrow A \rightarrow C$.

5.1.3 Implementation Strategies for Trees

Trees are typically implemented using node-based structures. A **Node** struct contains data and pointers to child nodes:

```
struct Node {
    int data;
    Node* left;
    Node* right;
};
```

Binary Search Trees (BSTs)

A BST enforces the property that for any node, all left descendants have values less than the node, and all right descendants have values greater. Insertion and deletion maintain this property: - **Insertion:** Start at the root. If the new value is less than the current node, move left; otherwise, move right. Insert as a leaf. - **Deletion:** Three cases arise: 1. Node is a leaf: Remove it. 2. Node has one child: Replace it with its child. 3. Node has two children: Replace it with its inorder successor (smallest node in the right subtree) and delete the successor.

Balanced Trees

Unbalanced BSTs can degrade to linked lists in the worst case. Balanced trees, such as AVL trees and red-black trees, maintain height $O(\log n)$ through rotations during insertions and deletions. For example, AVL trees use balance factors (differences in subtree heights) to trigger rotations.

5.1.4 Applications of Trees

Trees are widely used in: - **File Systems:** Hierarchical organization of files and directories. - **Expression Parsing:** Abstract syntax trees represent mathematical expressions. - **Database Indexing:** B-trees enable efficient disk-based searches. - **Network Routing:** Spanning trees optimize path selection.

5.2 Conclusion

Trees provide a flexible framework for organizing hierarchical data. Understanding traversal algorithms and implementation strategies is critical for designing efficient data structures. Balanced trees, in particular, ensure optimal performance for dynamic operations.

Tutorial Questions

1. Define a binary tree and explain the difference between a full binary tree and a complete binary tree.
2. Describe the steps for performing an inorder traversal of a binary tree.
3. How does a BST ensure efficient search operations, and what is its worst-case time complexity?
4. Explain the process of deleting a node with two children in a BST.
5. What is the purpose of rotations in AVL trees, and how do they maintain balance?
6. Compare the memory allocation strategies for static arrays and dynamic trees.
7. Write a recursive function to calculate the height of a binary tree.
8. Discuss one application of trees in computer science and explain its relevance.